

第 6 章：Tailwind CSS

實用優先的現代網頁設計

「Tailwind 的簡寫不只是為了偷懶，它是透過『編譯技術』換取了『極小的體積』，並透過『規範命名』換取了『系統的一致性』。」

6.1 為何選擇 Tailwind CSS ?

傳統 CSS 寫法：為元件命名 class (如 `.card`)，再定義樣式。

痛點：CSS 檔案龐大、命名困難、團隊風格不一致。

Tailwind 的解法 (Utility-First)：

直接把 CSS 屬性封裝成極短的工具類別，寫在 HTML 裡！

```
<!-- 傳統寫法 -->
```

```
<div class="chat-notification">...</div>
```

```
<!-- Tailwind 寫法 -->
```

```
<div class="p-6 max-w-sm mx-auto bg-white rounded-xl shadow-lg">...</div>
```

6.1.1 Tailwind 兩大優勢

1. 透過「編譯技術」換取極小體積：

- 使用 Tree-Shaking 技術，正式部署時只保留「有用到」的 class，移除多餘樣式，檔案經常只有幾十 KB。

2. 透過「規範命名」換取系統一致性：

- 預先定義好整套設計系統（調色盤、字級、間距比例）。
- 團隊間不需再通靈 `.text-gray-700` 是哪種灰，也不需爭吵 `p-4` 該是多少像素。

6.1 隨堂測驗 🙌

Q: 在一般教學或開發初期，我們會使用哪種方式最快引入 Tailwind CSS ？

- A. 下載原始碼安裝
- B. 使用 npm 安裝與 PostCSS 設定
- C. 引入 CDN `<script src="https://cdn.tailwindcss.com"></script>`
- D. 掛載 jQuery 套件

6.1 隨堂測驗 (解答)

解答：C. 引入 CDN

- 開發初期使用 CDN 能在瀏覽器直接動態生成樣式，隨寫隨看。
- 但正式上線 (Production) 時，為求效能與 Tree-Shaking，必須改用 CLI 或建置工具 (如 Vite) 編譯。

6.2 基礎觀念：顏色、文字與間距

👉 [查看互動 Demo 與原始碼](#)

- **顏色系統：** {屬性}-{顏色}-{色階} (如 `bg-blue-500` 、 `text-red-200`)。支援任意值：`bg-[#1da1f2]`。
- **排版字體：**
 - 大小：`text-sm` , `text-xl` , `text-4xl`
 - 粗細：`font-bold` , `font-black`
- **間距系統：**
 - `p-` (Padding), `m-` (Margin)
 - 1 單位 = `0.25rem` (4px) 。 `p-4` 即為 16px 。
 - 對稱設定：`px-4` (水平), `py-2` (垂直) 。

6.2 隨堂測驗 🙋

Q: 如果我想設定元素的 padding 為「水平 16px，垂直 8px」，應該使用哪組 Tailwind class ? (假設 1 單位 = 4px)

- A. `p-16 p-8`
- B. `px-4 py-2`
- C. `padding-x-4 padding-y-2`
- D. `pl-4 pr-4 pt-2`

6.2 隨堂測驗 (解答)

解答：B. `px-4 py-2`

- `x` 代表 X 軸 (水平：Left & Right)。 `4` 單位 = 16px。
- `y` 代表 Y 軸 (垂直：Top & Bottom)。 `2` 單位 = 8px。
- 當然你也可以寫 `pt-2 pb-2 pl-4 pr-4`，但 `px-4 py-2` 簡潔許多。

6.3 佈局與排版：Flexbox 與 Grid

👉 [查看互動 Demo 與原始碼](#)

透過最外層容器的 Class 就能快速決定內部排列方式！

- **Flexbox:**

- `flex`：啟動 Flex 佈局。
- `items-center` (垂直置中), `justify-between` (兩端對齊)。

- **Grid:**

- `grid`：啟動網格。
- `grid-cols-3` (設定三欄), `gap-8` (設定間距)。
- `col-span-2` (讓子元素跨兩欄)。

6.3 隨堂測驗 🙌

Q: 在 Flex 容器中，哪一個 class 可以讓所有的子元素「水平平均分散（兩端貼齊）」？

- A. `items-center`
- B. `justify-center`
- C. `justify-between`
- D. `align-items-stretch`

6.3 隨堂測驗 (解答)

解答：C. `justify-between`

- `justify-between` 代表主軸 (Main Axis) 上的可用空間會平均分配在項目之間，第一個項目貼齊起點，最後一個貼齊終點。
- 等同於原生 CSS 的 `justify-content: space-between;`。

6.4 視覺精修：邊框、圓角與陰影

👉 [查看互動 Demo 與原始碼](#)

只要幾個 class，立刻擁有現代化 UI 質感。

- **圓角 (Rounded)**： `rounded-md` , `rounded-3xl` , `rounded-full` 。
- **陰影 (Shadow)**： `shadow-sm` , `shadow-xl` , `shadow-2xl` 營造立體深度。
- **光環 (Ring)**： `ring-2 ring-blue-500` ，適合用來製作按鈕的 Focus 狀態，或是代替邊框 (Border) 做出更輕盈的效果。
- **圖片設定**： `aspect-video` (16:9), `object-cover` 。

6.4 隨堂測驗 🙌

Q: 如何在 Tailwind 中將一張人物大頭貼變為「完全的圓形」？

- A. `border-radius: 50%;`
- B. `rounded-circle`
- C. `rounded-full`
- D. `rounded-3xl`

6.4 隨堂測驗 (解答)

解答：C. `rounded-full`

- `rounded-full` 等同於原生 CSS 的 `border-radius: 9999px;`，用在正方形元素上就會變成正圓形，非常適合用於大頭貼 (Avatar) 或藥丸狀的按鈕 (Pill buttons)。

6.5 響應式設計 (Responsive Design)

👉 [查看互動 Demo 與原始碼](#)

Tailwind 採用 **Mobile First (手機優先)** 策略！

- 無前綴的 class (`w-full`)：預設套用到手機版。
- 帶有斷點前綴的 class (`md:w-1/2`)：螢幕大於該斷點時，會「覆蓋」預設值。

前綴	最小寬度	對應裝置
<code>sm:</code>	640px	大手機 / 平板直向
<code>md:</code>	768px	平板橫向
<code>lg:</code>	1024px	筆電 / 電腦

6.5 常見響應式技巧

- 結構變化：

手機版直向堆疊，桌機版橫向排列：`flex-col md:flex-row`

- 元素隱藏：

手機版不顯示側邊欄，桌機版才顯示：`hidden md:block`

- 網格變化：

手機 1 欄、平板 2 欄、桌機 3 欄：`grid-cols-1 md:grid-cols-2 lg:grid-cols-3`

6.6 互動與狀態 (States & Transitions)

👉 查看[互動 Demo](#) 與原始碼

動態效果與互動狀態也能用 Utility-First 搞定。

- **hover:** (懸停) : `hover:bg-blue-700`
- **focus:** (選取聚焦) : `focus:ring-2`
- **active:** (點擊當下) : `active:scale-95` (點擊時按鈕微微縮小)
- **group-hover:** : 標記外層 container 為 `group` , 當滑鼠懸停於外層時, 觸發內部元素的變色或位移。
- **過渡動畫** : 別忘了加上 `transition-all duration-300` 使動畫平滑。

6.7 進階技巧：使用 @apply 自定義 Class

👉 [查看互動 Demo 與原始碼](#)

當 Class 寫得太長變成 "Class Soup"，或同一個樣式（如按鈕）在各處出現多次時怎麼辦？

透過 `<style type="text/tailwindcss">`，使用 `@apply` 提取元件：

```
.btn-primary {  
  @apply bg-blue-600 text-white font-bold py-3 px-6 rounded-full hover:bg-blue-700 transition-all;  
}
```

然後在 HTML 裡直接套用：`<button class="btn-primary">確認</button>`。

6.7 隨堂測驗 🙌

Q: 關於 `@apply` 的使用時機，以下敘述何者錯誤？

- A. 解決 HTML 原始碼中 class 過長難以閱讀的問題。
- B. 用來封裝在多個頁面都會「重複使用」的 UI 元件（如 `.card`）。
- C. 為了保持 HTML 乾淨，每個元素都應該用 `@apply` 抽出獨立的 class 命名。
- D. 使用 CDN 時，必須加上 `type="text/tailwindcss"` 才能解析它。

6.7 隨堂測驗 (解答)

解答：C. 每個元素都應該用 `@apply` 抽出獨立命名

- 這是**錯誤的**！如果每個元素都自己命名再用 `@apply`，那就退回到了傳統 CSS 時代的作法，不僅增加命名負擔，也拋棄了 Tailwind「直接在 HTML 中建構 UI」的敏捷精神。
- **原則**：只抽離**高度重複**的邏輯元件。

6.9 實戰指南：開發輔助工具

不要死背 class，善用工具才是王道！

- **官網字典 (tailwindcss.com/docs)**：使用 `Ctrl+K` 搜尋原生的 CSS 屬性，快速反查。
- **VS Code: Tailwind CSS IntelliSense**：
 - 自動補全 (Autocomplete) 與色塊預覽。
 - 滑鼠懸停即時查閱原生 CSS 數值。
- **AI 輔助開發**：使用 ChatGPT / Copilot 生成元件或加上深色模式 (`dark:bg-gray-900`)。

6.11 部署專案：使用 GitHub Pages

寫好的靜態網頁如何讓全世界看見？

GitHub Pages 提供免費、快速的靜態網頁代管服務。

1. **建立 Repo**：在 GitHub 開新專案 (Public)。
2. **上傳檔案**：將你的 `index.html` 丟進去並 Commit。
3. **開啟服務**：進入專案 `Settings` > `Pages`。
4. **設定來源**：選擇 `Deploy from a branch`，Branch 選擇 `main`，按下 `Save`。
5. **等待一分鐘**：專屬網址 (`帳號.github.io/專案名稱`) 就誕生了！

Q & A

恭喜！你已經掌握了現代最強大前端框架的核心兵器。
準備好打造充滿質感的個人網站了嗎？